Aprendizagem 2022/23

Homework I – Group 74

(ist103363, ist103796)

I. Pen-and-paper

1

a)

```python
import numpy as np

# Data points and targets
X = np.array([[0.7, -0.3], [0.4, 0.5], [-0.2, 0.8], [-0.4, 0.3]])
y = np.array([0.8, 0.6, 0.3, 0.3])

# Centers
c1 = np.array([0, 0])
c2 = np.array([1, -1])
c3 = np.array([-1, 1])

# Compute RBF features for each data point
phi0 = np.ones(X.shape[0])
phi1 = np.exp(-np.linalg.norm(X - c1, axis=1)**2 / 2)
phi2 = np.exp(-np.linalg.norm(X - c2, axis=1)**2 / 2)
phi3 = np.exp(-np.linalg.norm(X - c3, axis=1)**2 / 2)

# Create the design matrix Φ
Φ = np.column_stack((phi0, phi1, phi2, phi3))

# Regularization parameter
λ = 0.1

# Calculate the weights using Ridge regression formula
I = np.identity(4)
w = np.linalg.inv(Φ.T @ Φ + λ * I) @ Φ.T @ y

# Print the weights
print(w)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[tteixeira@tt-msi ~]$ /bin/python /home/tteixeira/Desktop/Ano3/Semestre1/Apre/H
[ 0.33914267  0.19945264  0.40096085 -0.29599936]
```

Ridge regression : $y(x) = 0{,}33914267 + 0{,}19945264\,x_1 + 0{,}40096085\,x_2 - 0{,}29599936\,x_3$

b)

$$RMSE(\hat{y}, y) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}$$

```python
# Make predictions on the training data
y_pred = Φ @ w

# Calculate residuals
residuals = y - y_pred

# Calculate RMSE
rmse = np.sqrt(np.mean(residuals**2))

# Print the RMSE
print("Training RMSE:", rmse)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[tteixeira@tt-msi ~]$ /bin/python /home/tteixeira/De
Training RMSE: 0.06508238153393466
```

②

Forward Equations:

$$\underline{z}^{[1]} = \underline{\underline{w}}^{[1]} \underline{x}^{[0]} + \underline{b}^{[1]}$$

$$\underline{x}^{[1]} = \tanh\left(\frac{1}{2}\underline{z}^{[1]} - 2\right)$$

$$\underline{z}^{[2]} = \underline{\underline{w}}^{[2]} \underline{x}^{[1]} + \underline{b}^{[2]}$$

$$\underline{x}^{[2]} = \tanh\left(\frac{1}{2}\underline{z}^{[2]} - 2\right)$$

$$\underline{z}^{[3]} = \underline{\underline{w}}^{[3]} \underline{x}^{[2]} + \underline{b}^{[3]}$$

$$\underline{x}^{[3]} = \tanh\left(\frac{1}{2}\underline{z}^{[3]} - 2\right)$$

$$E(\underline{t}, \underline{x}^{[3]}) = \frac{1}{2}\left(\underline{t} - \underline{x}^{[3]}\right)^2$$

$$\frac{d \tanh\left(\frac{1}{2}x - 2\right)}{dx} = \frac{1}{2}\left(1 - \tanh^2\left(\frac{1}{2}x - 2\right)\right)$$

$$\left.\begin{aligned} \frac{\partial E}{\partial \underline{\underline{w}}^{[l]}} &= \underline{\delta}^{[l]} \underline{x}^{[l-1]^T} \\ \frac{\partial E}{\partial b^{[l]}} &= \underline{\delta}^{[l]} \end{aligned}\right\} \quad l = 1, \dots, L-1 \text{ último nível}$$

$$\underline{\delta}^{[L]} = \frac{\partial E(\underline{t}, \underline{x}^{[L]})}{\partial \underline{x}^{[L]}} \odot f_a'(\underline{z}^{[L]}) \Rightarrow L$$

$$\delta^{[l]} = \underline{\underline{w}}^{[l+1]^T} \underline{\delta}^{[l+1]} \odot f_a'(\underline{z}^{[l]}) \Rightarrow l = 1, \dots, L-1$$

$$\underline{\underline{w}}^{[l]^{(m+1)}} = \underline{\underline{w}}^{[l]^{(m)}} - \eta \underbrace{\frac{\partial E}{\partial \underline{\underline{w}}^{[l]}}}_{\sum_K \frac{\partial E(\underline{t}_K, \underline{x}_K^{[L]})}{\partial \underline{\underline{w}}^{[l]}}}$$

$$E(\underline{t}, \underline{x}^{[L]}) = \frac{1}{2} \sum_K \left(\underline{t}_K - \underline{x}_K^{[L]}\right)^2$$

```python
import numpy as np

W1 = np.array([[1,1,1,1],[1,1,2,1],[1,1,1,1]])
W2 = np.array([[1,4,1],[1,1,1]])
W3 = np.array([[1,1],[3,1],[1,1]])

b1 = np.array([1,1,1])
b2 = np.array([1,1])
b3 = np.array([1,1,1])



def layer1(x):
  return np.dot(W1,x)+b1

def layer2(x):
  return np.dot(W2,x)+b2

def layer3(x):
  return np.dot(W3,x)+b3
```

```python
def act_f(x):
    return np.tanh(0.5*x - 2)
```

```python
def nn(x):
  a1 = layer1(x)
  h1 = act_f(a1)
  a2 = layer2(h1)
  h2 = act_f(a2)
  a3 = layer3(h2)
  h3 = act_f(a3)
  return x,a1,h1,a2,h2,a3,h3
```

```
x1 = [1,1,1,1]
x2 = [1,0,0,-1]
z1 = [-1,1,-1]
z2 = [1,-1,-1]
```

```
def calc_grad(x, z):

  x, a1, h1, a2, h2, a3, h3 = nn(x)
  print(h3)
  delta3 =(h3 - z) * (1 / (2 * np.cosh(2 - 0.5 * a3)**2))
  grad3 = np.outer(delta3,h2)
  delta2 = W3.T @ delta3 * (1 / (2 * np.cosh(2 - 0.5 * a2)**2))
  grad2 = np.outer(delta2, h1)
  delta1 = W2.T @ delta2 * (1 / (2 * np.cosh(2 - 0.5 * a1)**2))
  grad1 = np.outer(delta1, x)

  return np.array(delta1), np.array(grad1), np.array(delta2), np.array(grad2), np.array(delta3), np.array(grad3)
```

```
samples = [x1,x2]
targets = [z1, z2]
for i in range(2):
  if(i==0):
   delta1, grad1, delta2, grad2, delta3, grad3 = calc_grad(samples[i], targets[i])
  else:
    d1, g1, d2, g2, d3, g3 = calc_grad(samples[i], targets[i])
    delta1 += d1
    grad1 += g1
    delta2 += d2
    grad2 += g2
    delta3 += d3
    grad3 += g3

lr = 0.1

b1_new = b1 - lr * delta1
b2_new = b2 - lr * delta2
b3_new = b3 - lr * delta3

W1_new = W1 - lr * grad1
W2_new = W2 - lr * grad2
W3_new = W3 - lr * grad3

print(f'''New params:\n
W1:\n{W1_new}\nW2:\n{W2_new}\nW3:\n{W3_new}\nb1: {b1_new}\nb2: {b2_new}\nb3: {b3_new}\n''')
```

```
[-0.91590016 -0.80493961 -0.91590016]
[-0.98652085 -0.9981635  -0.98652085]
New params:

W1:
[[1.0187207  1.01871904 1.01871904 1.01871737]
 [1.03358917 1.03358719 2.03358719 1.03358521]
 [1.0187207  1.01871904 1.01871904 1.01871737]]
W2:
[[1.01730444 4.02851932 1.01730444]
 [1.00467751 1.00771893 1.00467751]]
W3:
[[0.99703633 0.9977484 ]
 [3.01431372 0.98168546]
 [0.99971282 1.00040847]]
b1: [1.0187207  1.03358917 1.0187207 ]
b2: [1.0374494  1.01017306]
b3: [1.00198208 1.03177312 0.99930442]
```

Aprendizagem 2022/23

Homework I – Group 74

(ist103363, ist103796)

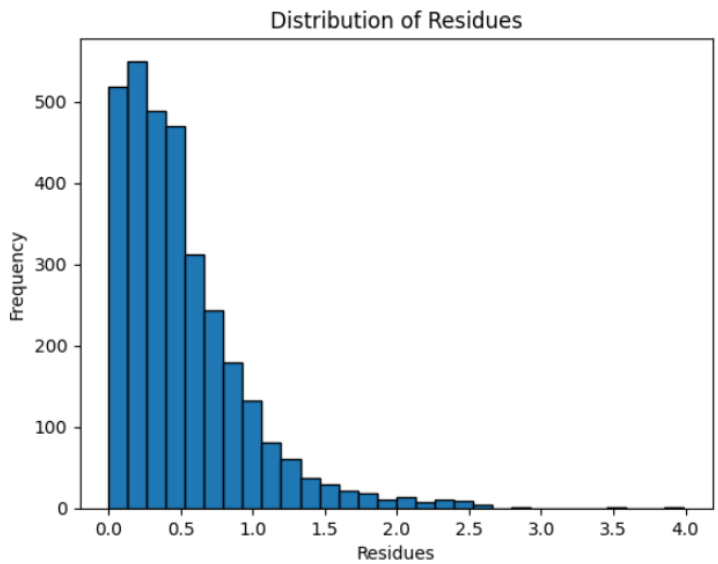
II. Programming and critical analysis

#1. Learning MLP classifier

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
import numpy as np
import matplotlib.pyplot as plt

data = pd.read_csv("winequality-red.csv", delimiter = ';')

X = data.drop(columns=['quality'])
y = data['quality']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

num_runs = 10
residues = []
rmse_sum = 0

for seed in range(1, num_runs + 1):
    mlp = MLPRegressor(hidden_layer_sizes=(10, 10), activation='relu',
                       random_state=seed, early_stopping=True, validation_fraction=0.2)
    mlp.fit(X_train, y_train)
    y_pred = mlp.predict(X_test)
    residues.extend(np.abs(y_test - y_pred))

    rmse_sum += np.sqrt(mean_squared_error(y_test, y_pred))

rmse_mean = rmse_sum/10
```

#Plotting absolute distribution of residues

```
n, bins, patches = plt.hist(residues, bins=30, edgecolor='k')
plt.xlabel('Residues')
plt.ylabel('Frequency')
plt.title('Distribution of Residues')
plt.show()
```



#2 Checking the impact on MAE, of rounding and bounding the results of model's output

```
rmse_sum = 0
mae_original_sum = 0
mae_rounded_sum = 0
mae_bounded_sum = 0

for seed in range(1, num_runs + 1):
    mlp = MLPRegressor(hidden_layer_sizes=(10, 10), activation='relu', random_state=seed,
                       early_stopping=True, validation_fraction=0.2)
    mlp.fit(X_train, y_train)
    y_pred = mlp.predict(X_test)
    y_pred_rounded = np.round(y_pred)
    y_pred_bounded = np.clip(y_pred, 3, 8)

    mae_original_sum += mean_absolute_error(y_test, y_pred)
    mae_rounded_sum += mean_absolute_error(y_test, y_pred_rounded)
    mae_bounded_sum += mean_absolute_error(y_test, y_pred_bounded)

mae_original = mae_original_sum/10

mae_rounded = mae_rounded_sum/10
mae_bounded = mae_bounded_sum/10

print(f"MAE without rounding or bounding: {mae_original}")
print(f"MAE with rounding: {mae_rounded}")
print(f"MAE with bounding: {mae_bounded}")
```

```
MAE without rounding or bounding: 0.5097171955009515
MAE with rounding: 0.43875000000000003
MAE with bounding: 0.5097171955009515
```

As we can see, by rounding the results, MAE decreases. This is likely because our labels are integers, while the output of our model is a real number. Therefore, even when our model predicts correctly (the rounded number matches a true label), the error increases. This is unless our model outputs the exact correct integer (which is very unlikely). By rounding our output and then calculating the loss function, the only cases that will impact our error are those in which our model predicts the wrong label for the given features.

By bounding, we didn't observe any differences in MAE. This is because our model didn't predict any numbers that fall outside the domain of the problem.

#3. Replacing early stopping by a well-defined number of iterations, and checking the impact on RMSE

```
num_iterations = [20, 50, 100, 200]
rmse_values = []
rmse_s = 0
for iterations in num_iterations:
    for seed in range(1, num_runs + 1):
      mlp = MLPRegressor(hidden_layer_sizes=(10, 10), activation='relu', random_state=seed, max_iter=iterations)
      mlp.fit(X_train, y_train)
      y_pred = mlp.predict(X_test)
      rmse_s += np.sqrt(mean_squared_error(y_test, y_pred))

    rmse_values.append(rmse_s/10)
    rmse_s = 0

plt.plot(num_iterations, rmse_values, marker='o')
plt.xlabel('Number of Iterations')
plt.ylabel('Root Mean Square Error (RMSE)')
plt.title('RMSE vs. Number of Iterations')
plt.grid(True)
plt.show()

print(f'''ERRORS:\n
20 iterations: {rmse_values[0]}\n
50 iterations: {rmse_values[1]}\n
100 iterations: {rmse_values[2]}\n
200 iterations: {rmse_values[3]}\n
Stopping depending on validation set: {rmse_mean}\n''')
```

#4. Commenting the results. In this case, replacing early stopping worsens the performance of our model. Probably, this is because, in case of early stopping, train data set is 20% less then without early stopping, so in case of exact number of iterations, the model can learn more per iteration. As our model is still in the process of learning and underfitting even up to 200 iterations. It's worth noting that our model doesn't overfit, likely due to its relatively low number of parameters and training on a substantial amount of data compared to the number of model parameters.

#Pen and paper. 2.

```
import numpy as np

W1 = np.array([[1,1,1,1],[1,1,2,1],[1,1,1,1]])
W2 = np.array([[1,4,1],[1,1,1]])
W3 = np.array([[1,1],[3,1],[1,1]])

b1 = np.array([1,1,1])
b2 = np.array([1,1])
b3 = np.array([1,1,1])



def layer1(x):
  return np.dot(W1,x)+b1

def layer2(x):
  return np.dot(W2,x)+b2

def layer3(x):
  return np.dot(W3,x)+b3

def act_f(x):
    return np.tanh(0.5*x - 2)


def nn(x):
  a1 = layer1(x)
  h1 = act_f(a1)
  a2 = layer2(h1)
  h2 = act_f(a2)
  a3 = layer3(h2)
  h3 = act_f(a3)
  return x,a1,h1,a2,h2,a3,h3
```

```
x1 = [1,1,1,1]
x2 = [1,0,0,-1]
z1 = [-1,1,-1]
z2 = [1,-1,-1]
```

```
def calc_grad(x, z):

  x, a1, h1, a2, h2, a3, h3 = nn(x)
  print(h3)
  delta3 =(h3 - z) * (1 / (2 * np.cosh(2 - 0.5 * a3)**2))
  grad3 = np.outer(delta3,h2)
  delta2 = W3.T @ delta3 * (1 / (2 * np.cosh(2 - 0.5 * a2)**2))
  grad2 = np.outer(delta2, h1)
  delta1 = W2.T @ delta2 * (1 / (2 * np.cosh(2 - 0.5 * a1)**2))
  grad1 = np.outer(delta1, x)

  return np.array(delta1), np.array(grad1), np.array(delta2), np.array(grad2), np.array(delta3), np.array(grad3)
```

```
samples = [x1,x2]
targets = [z1, z2]
for i in range(2):
  if(i==0):
   delta1, grad1, delta2, grad2, delta3, grad3 = calc_grad(samples[i], targets[i])
  else:
    d1, g1, d2, g2, d3, g3 = calc_grad(samples[i], targets[i])
    delta1 += d1
    grad1 += g1
    delta2 += d2
    grad2 += g2
    delta3 += d3
    grad3 += g3

lr = 0.1

b1_new = b1 - lr * delta1
b2_new = b2 - lr * delta2
b3_new = b3 - lr * delta3

W1_new = W1 - lr * grad1
W2_new = W2 - lr * grad2
W3_new = W3 - lr * grad3

print(f'''New params:\n
W1:\n{W1_new}\nW2:\n{W2_new}\nW3:\n{W3_new}\nb1: {b1_new}\nb2: {b2_new}\nb3: {b3_new}\n''')
```

```
[-0.91590016 -0.80493961 -0.91590016]
[-0.98652085 -0.9981635  -0.98652085]
New params:

W1:
[[1.0187207  1.01871904 1.01871904 1.01871737]
 [1.03358917 1.03358719 2.03358719 1.03358521]
 [1.0187207  1.01871904 1.01871904 1.01871737]]
W2:
[[1.01730444 4.02851932 1.01730444]
 [1.00467751 1.00771893 1.00467751]]
W3:
[[0.99703633 0.9977484 ]
 [3.01431372 0.98168546]
 [0.99971282 1.00040847]]
b1: [1.0187207  1.03358917 1.0187207 ]
b2: [1.0374494  1.01017306]
b3: [1.00198208 1.03177312 0.99930442]
```

END